Relatório Completo: Arquitetura de Solução para Aplicativo de Monitoramento Hospitalar por Geolocalização

Table of Contents

- Resumo Executivo
- 1. Visão Geral da Solução
- 2. Arquitetura da Solução
- 3. Análise Comparativa de Tecnologias
- 4. Análise de Riscos e Mitigações
- 5. Recomendações para MVP Enxuto
- 6. Exemplos Práticos de Implementação
- 7. Estratégias de Rollback e Controle de Custos
- Deploy versionado
- Rollback se necessário
- Atualização de serviço
- Desligar recursos dev/staging após expediente
  - 8. Segurança, Privacidade e LGPD
  - 9. Roadmap de Evolução
  - 10. Considerações Finais e Recomendações

Resumo Executivo

Este relatório apresenta uma arquitetura completa para desenvolvimento de um aplicativo de monitoramento hospitalar baseado em geolocalização, incluindo coleta de feedbacks dos usuários. A solução foi projetada para mapear o tempo de permanência dos pacientes em hospitais, identificar níveis de lotação e coletar avaliações sobre a qualidade do atendimento.

1. Visão Geral da Solução

1.1 Objetivo Principal

Desenvolver um aplicativo que permita:

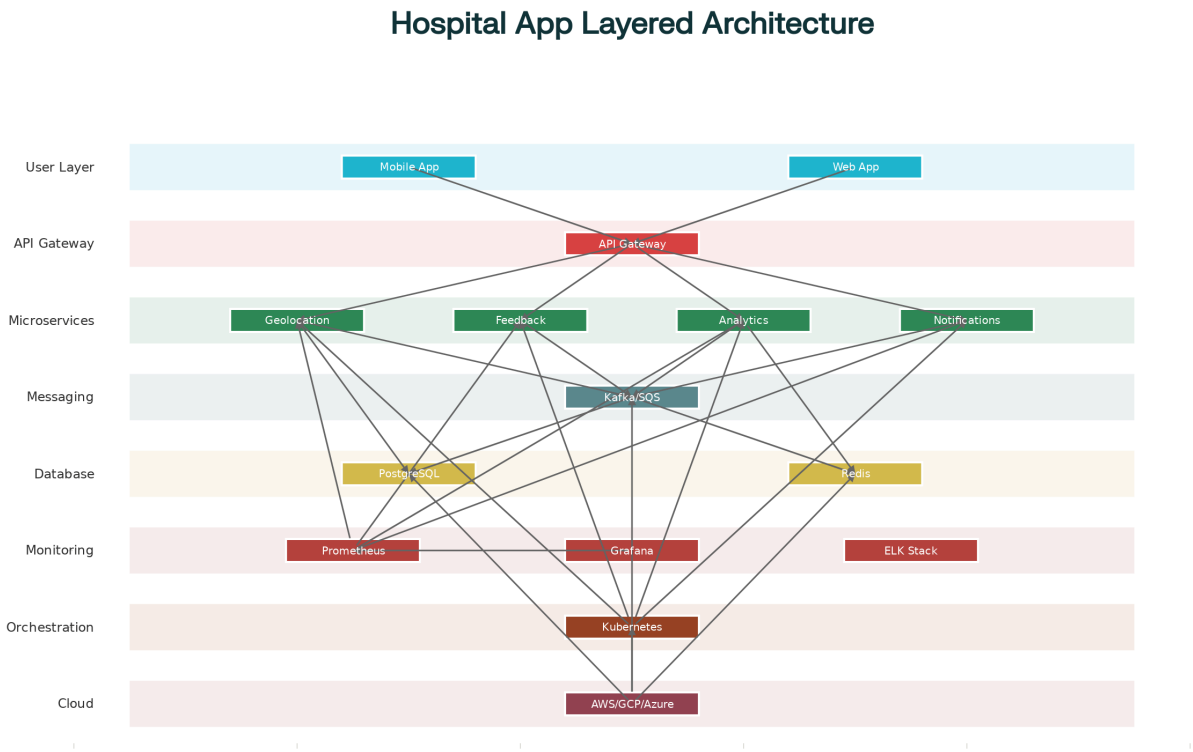
- **Mapeamento de usuários por geolocalização** em ambientes hospitalares
- **Cálculo do tempo de permanência** em cada setor do hospital
- **Identificação de lotação** e tempos de espera
- **Coleta de feedbacks** sobre experiência do atendimento
- **Analytics em tempo real** para gestão hospitalar

1.2 Funcionalidades Principais

- Detecção automática de entrada/saída via GPS/BLE/beacons
- Registro de permanência por especialidade médica
- Sistema de feedback estruturado (atendimento, especialidade, disponibilidade médica)
- Dashboards em tempo real para gestão hospitalar
- Analytics para otimização de fluxos

2. Arquitetura da Solução

2.1 Visão Geral da Arquitetura em Camadas



A arquitetura é baseada em microsserviços, integração cloud-native, boas práticas DevOps e forte ênfase em segurança, compondo-se de:

Aplicativo Mobile/Web

- Interface React (Web) e React Native (Mobile)
- Integração com sensores de localização (GPS, BLE, beacons)
- Sistema de mapas indoor para navegação
- Interface simplificada para coleta de feedbacks

Backend Distribuído (Microserviços)

- **API Gateway**: Spring Cloud Gateway (autenticação, roteamento, rate limiting)
- **Location Service**: Java Quarkus (processamento geoespacial)
- **Feedback Service**: Spring WebFlux/Kotlin (coleta e análise de avaliações)
- **Analytics Service**: Java Quarkus (relatórios e métricas)

- **Notification Service**: Spring WebFlux (notificações push)

Infraestrutura de Dados

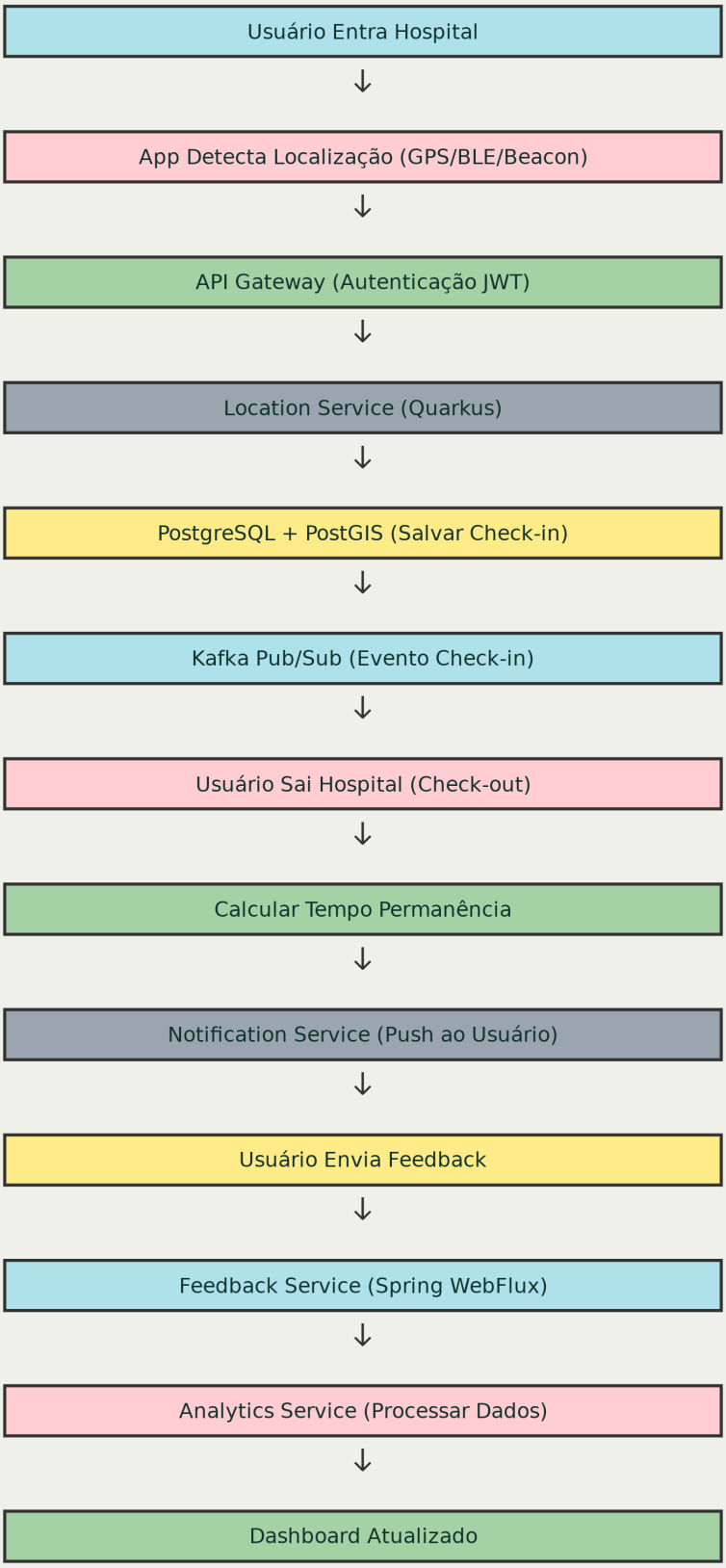
- **PostgreSQL + PostGIS**: Dados principais e geoespaciais
- **Redis**: Cache de sessões e dados temporários
- **Apache Kafka**: Mensageria assíncrona entre serviços

Monitoramento e Observabilidade

- **Prometheus + Grafana**: Métricas e dashboards
- **ELK Stack**: Logs centralizados
- **Jaeger**: Tracing distribuído

2.2 Fluxo Operacional do Sistema

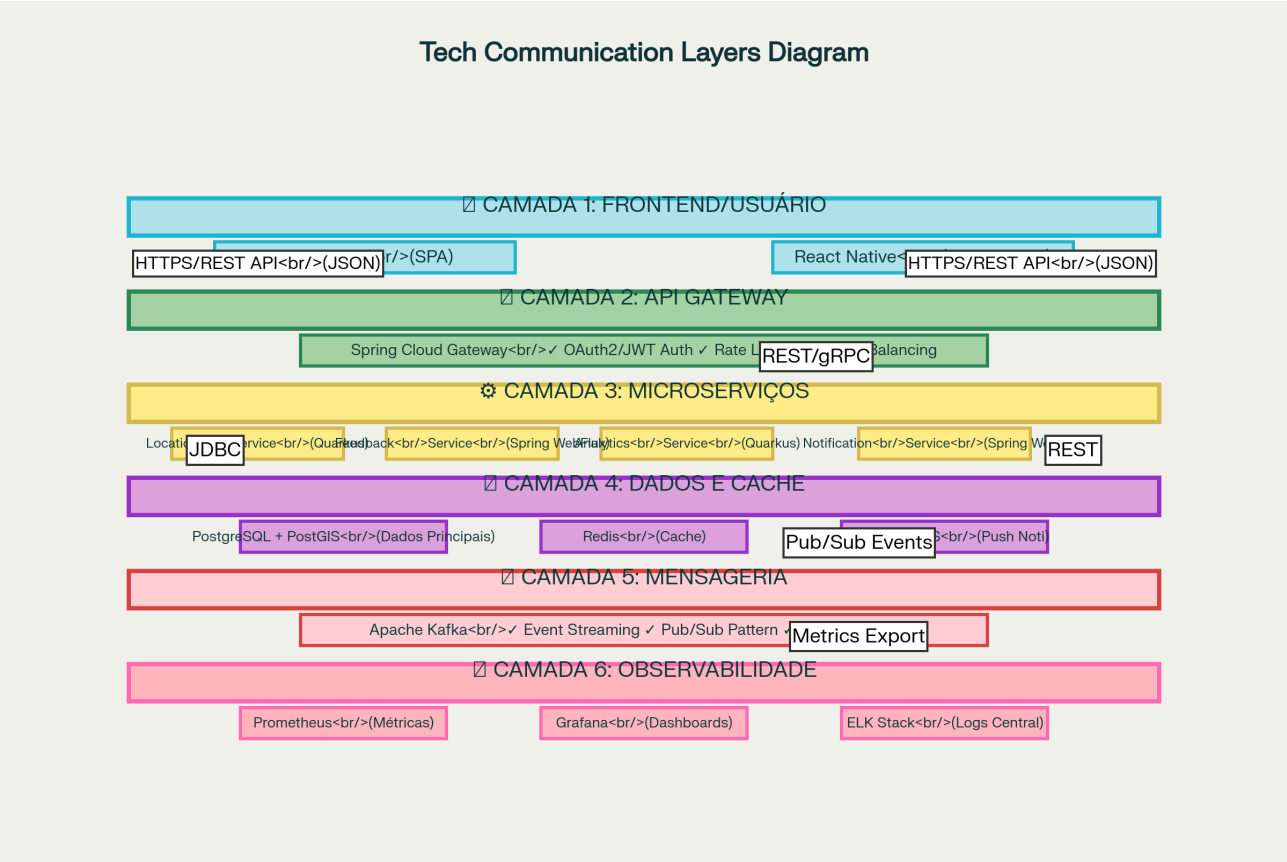
Hospital Check-in System Flow



O fluxo operacional segue as seguintes etapas:

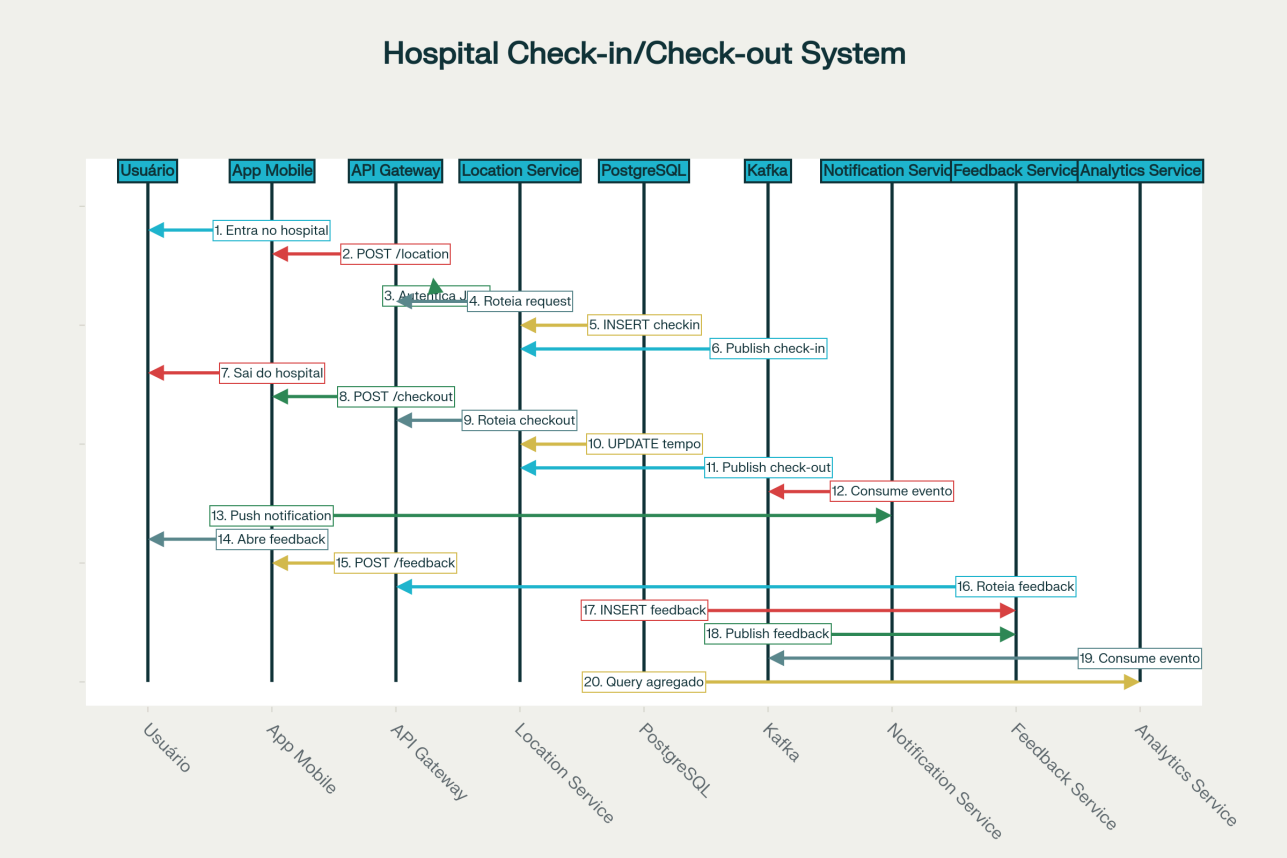
1. **Check-in Automático**: App detecta entrada via GPS/BLE/beacons
2. **Processamento**: Location Service calcula permanência e registra dados
3. **Eventos Assíncronos**: Kafka distribui eventos para outros serviços
4. **Feedback**: Notificação push solicita avaliação pós check-out
5. **Analytics**: Agregação de dados para dashboards gerenciais

2.3 Comunicação Entre Tecnologias



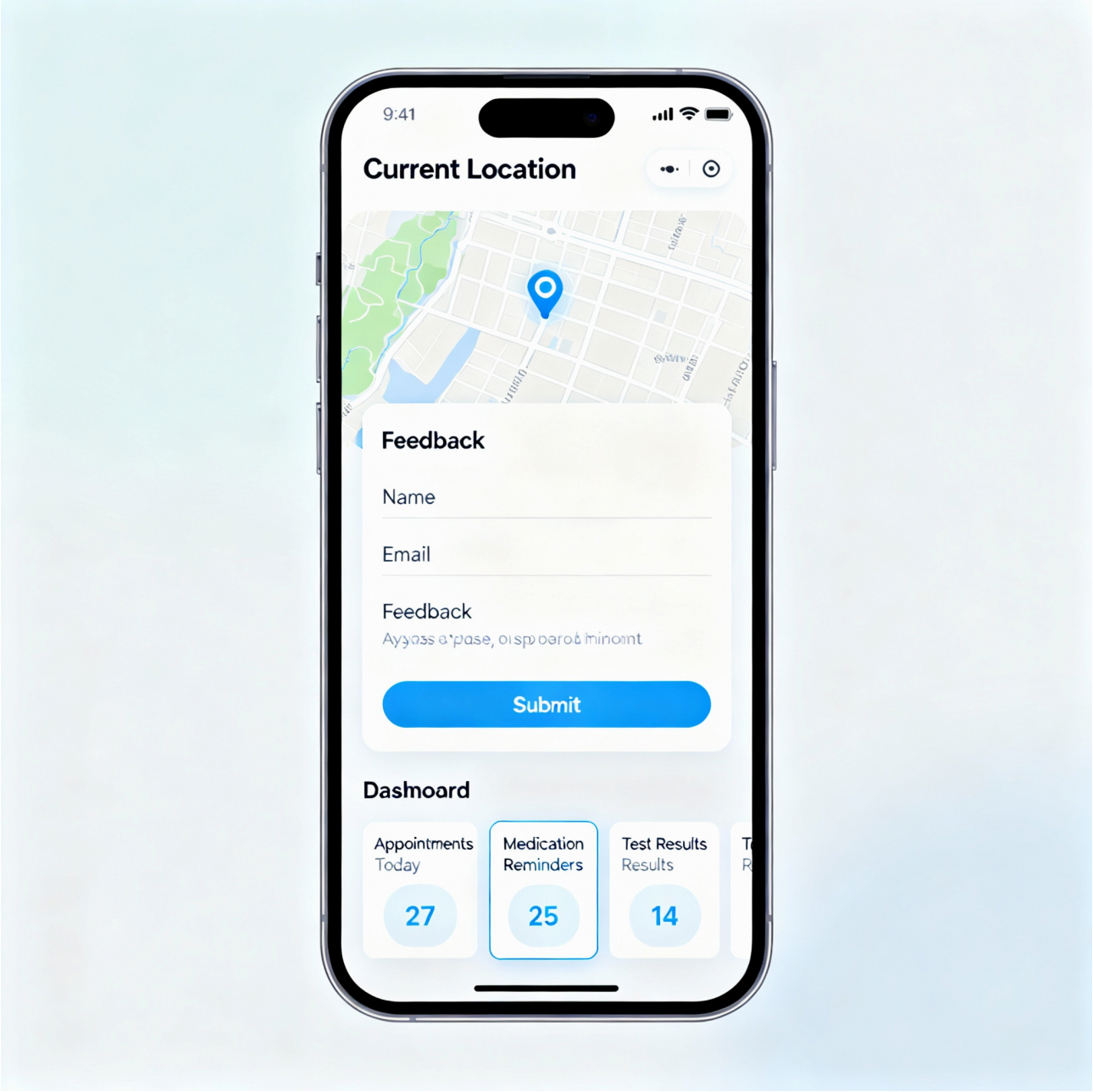
O diagrama acima ilustra como cada componente se comunica através de diferentes protocolos e camadas de abstração, garantindo baixo acoplamento e alta escalabilidade.

2.4 Sequência Temporal de Interações



Este diagrama de sequência mostra a ordem temporal das interações entre todos os componentes, desde a entrada do usuário até a atualização dos dashboards gerenciais.

2.5 Protótipo Visual da Interface



3. Análise Comparativa de Tecnologias

3.1 Matriz Comparativa Detalhada

Camada	Tecnologia Recomendada	Pontos Positivos	Pontos Negativos	Alternativas
Mobile/Web	React/React Native	Vasto ecossistema, rápida prototipagem, compartilhamento de código	Complexidade crescente, performance em devices antigos	Flutter, Vue.js, Angular
Geolocalização	Java Quarkus	Leve, cloud-native, baixo uso de memória	Menor base de profissionais experientes	Go, Node.js (NestJS), Rust

Camada	Tecnologia Recomendada	Pontos Positivos	Pontos Negativos	Alternativas
Backend Feedback	Spring WebFlux/Kotlin	Reatividade/escala, integração Java corporativa	Setup inicial complexo, alto uso de memória se mal configurado	Node.js, Python FastAPI
Banco de Dados	PostgreSQL + PostGIS + Redis	Robusto, ACID, excelente para dados geoespaciais	Limitações de escala horizontal, requer tuning	MongoDB, Cassandra, DynamoDB
Mensageria	Apache Kafka	Alto throughput, tolerante a falhas	Complexidade operacional, demanda infra robusta	RabbitMQ, AWS SQS
Orquestração	Docker + Kubernetes	Padrão de mercado, flexível	Curva de aprendizado alta, demanda DevOps experiente	AWS ECS, Azure AKS, GCP GKE

3.2 Justificativas das Escolhas

Java Quarkus/Spring WebFlux

- Escolhidos por integração com ambientes corporativos hospitalares
- Alta resiliência e suporte empresarial
- Comunidade ativa e documentação robusta
- **Contras**: Curva de aprendizado para times não familiarizados com Java

PostgreSQL + PostGIS

- Melhor solução para consultas geoespaciais complexas
- Suporte nativo a transações ACID
- **Contras**: Requer tuning cuidadoso para alta escala

Apache Kafka

- Excelente para eventos em tempo real e alta escalabilidade
- **Contras**: Complexidade operacional significativa

4. Análise de Riscos e Mitigações

4.1 Matriz Completa de Riscos

Categoria	Risco Potencial	Impacto	Probabilidade	Mitigação Detalhada
DevOps/K8s/Kafka	Falhas em deploy, downtime	Alto	Média-Alta	Automação CI/CD completa, Health Check por readiness/liveness probes, scripts de rollback via kubectl, backup automático
Escalabilidade	Sobrecarga de banco/PostgreSQL	Alto	Média	Auto-scaling DB, connection pool, Redis caching, sharding/replicação, testes de carga
Cloud Cost	Gastos fora do orçamento	Alto	Alta	Budget alerts, auto-stop dev/staging, tags obrigatórias, relatórios detalhados

Categoria	Risco Potencial	Impacto	Probabilidade	Mitigação Detalhada
Segurança/LGPD	Vazamento/danos legais	Muito Alto	Baixa-Média	Criptografia AES-256, anonimização, auditorias, testes de penetração
Performance	Latência, gargalos	Médio	Média	Stress test contínuo, monitoramento automático, profiling, filas assíncronas
Mobile UX	Falhas na localização	Médio	Média	Bibliotecas auditadas, testes em múltiplos devices, feedback visual
Adesão Feedback	Baixa adesão usuário	Médio-Alto	Alta	UI simplificada, gamificação, notificações automáticas
Observabilidade	Perda de logs/métricas	Médio	Baixa	Storage alternativo, redundância, alertas para gaps

5. Recomendações para MVP Enxuto

5.1 Simplificações Iniciais

- **Backend único** ao invés de microserviços completos
- **Cloud database gerenciado** (RDS, CloudSQL)
- **Mensageria simples** (SQS, RabbitMQ) ao invés de Kafka
- **ECS/Fargate** ao invés de Kubernetes
- **Monitoramento SaaS** (Datadog, CloudWatch)

5.2 Estrutura de Dados Básica

```
-- Tabelas principais do MVP
CREATE TABLE checkin (
    id SERIAL PRIMARY KEY,
    user_id VARCHAR,
    hospital_id VARCHAR,
    latitude DOUBLE PRECISION,
    longitude DOUBLE PRECISION,
    checkin_time TIMESTAMPTZ,
    checkout_time TIMESTAMPTZ
);

CREATE TABLE feedback (
    id SERIAL PRIMARY KEY,
    user_id VARCHAR,
    hospital_id VARCHAR,
    specialty VARCHAR,
    status VARCHAR,
    rating INT,
    comments TEXT,
    created_at TIMESTAMPTZ
);
```

6. Exemplos Práticos de Implementação

6.1 Pipeline CI/CD (GitHub Actions)

```
name: Deploy Hospital App
on:
  push:
    branches: [ main ]
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - name: Build Docker Image
      run: docker build -t hospital:v1 .
    - name: Deploy to ECS
      run: aws ecs update-service --cluster myCluster --service myService --force-new-depl
```

6.2 Health Check (Spring Boot)

```
@RestController
public class HealthController {

    @GetMapping("/health")
    public ResponseEntity&lt;Map&lt;String, String&gt;&gt; health() {
        Map&lt;String, String&gt; status = new HashMap&lt;&gt;();
        status.put("status", "UP");
        status.put("timestamp", Instant.now().toString());
        return ResponseEntity.ok(status);
    }
}
```

6.3 Integração com Mapbox (React)

```
import MapGL, { Marker } from "react-map-gl";

function HospitalMap({latitude, longitude}) {
  const [viewport, setViewport] = React.useState({
    latitude: latitude || -23.5505,
    longitude: longitude || -46.6333,
    zoom: 15
  });

  return (
    &lt;MapGL
      {...viewport}
      mapboxApiAccessToken={process.env.REACT_APP_MAPBOX_TOKEN}
      onViewportChange={setViewport}
    &gt;
      &lt;Marker latitude={latitude} longitude={longitude}&gt;
        <span>□</span>
      &lt;/Marker&gt;
    &lt;/MapGL&gt;
  );
}
```

6.4 Orquestração/Deploy (Kubernetes)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hospital-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hospital
  template:
    metadata:
      labels:
        app: hospital
    spec:
      containers:
      - name: hospital-backend
        image: 1234567890.dkr.ecr.sa-east-1.amazonaws.com/hospital:v1
        ports:
        - containerPort: 8080
        readinessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 10
          periodSeconds: 10
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 20
          periodSeconds: 20
```

7. Estratégias de Rollback e Controle de Custos

7.1 Planos de Rollback

Kubernetes

```bash
# Deploy versionado<a></a>
kubectl apply -f app-v2.yaml

# Rollback se necessário<a></a>
kubectl rollout undo deployment/hospital-app
```

AWS ECS

```bash
# Atualização de serviço<a></a>
aws ecs update-service --cluster myCluster --service myService --force-new-deployment
```

7.2 Controle de Custos Cloud

Scripts de Economia

```bash
# Desligar recursos dev/staging após expediente<a></a>
aws ec2 stop-instances --instance-ids i-xxxxxxxxxxxx
aws rds stop-db-instance --db-instance-identifier mydb-staging
```

Automação de Budgets

- Configurar alertas em 80%, 100% e 120% do orçamento
- Scripts de auto-stop para ambientes não produtivos
- Revisões mensais com equipe financeira

8. Segurança, Privacidade e LGPD

8.1 Requisitos de Conformidade

- **Criptografia AES-256** para dados em repouso
- **TLS 1.3** para comunicação em trânsito
- **Anonimização** de dados pessoais para analytics
- **Consentimento explícito** para coleta de localização
- **Portabilidade de dados** conforme LGPD
- **Trilha de auditoria** para todos os acessos

8.2 Controles de Segurança

- OAuth 2.0 + JWT para autenticação
- RBAC (Role-Based Access Control)
- API rate limiting e throttling
- Logs de auditoria centralizados

9. Roadmap de Evolução

Fase 1 - MVP (3-4 meses)

- Aplicativo web básico
- Backend unificado
- Funcionalidades core de localização e feedback
- Deploy em cloud gerenciada

Fase 2 - Expansão (6-8 meses)

- Aplicativo mobile nativo
- Microserviços completos
- Analytics avançados
- Integração com sistemas hospitalares

Fase 3 - IA e Predição (12+ meses)

- Machine Learning para predição de lotação
- Chatbot para atendimento
- Analytics preditivos
- Multi-tenancy para redes hospitalares

10. Considerações Finais e Recomendações

A arquitetura proposta oferece uma base sólida e escalável para o monitoramento hospitalar por geolocalização. As tecnologias escolhidas garantem:

- **Alta disponibilidade** e resiliência
- **Escalabilidade** para grandes volumes
- **Segurança** e conformidade LGPD
- **Flexibilidade** para evoluções futuras

10.1 Recomendações de Implementação

1. **Comece com MVP simplificado** para validar conceito
2. **Invista em equipe DevOps experiente** desde o início
3. **Implemente monitoramento** e observabilidade desde a primeira versão
4. **Mantenha foco na experiência do usuário** para garantir adesão
5. **Documente todos os processos** para facilitar manutenção

10.2 Checklist de Automação MVP

- [x] Deploy orientado a versão e rollback automatizado
- [x] Backup/resgate automático de banco de dados
- [x] Budget cloud definido e alertas automáticos ativos
- [x] Scripts de desligamento automático para ambientes não produtivos
- [x] Documentação da API publicada e atualizada (Swagger/OpenAPI)
- [x] Monitoramento básico (aplicação + infra) ativo
- [x] Healthchecks configurados nos endpoints

10.3 Próximos Passos

- Validação do MVP com hospitais piloto
- Definição de equipe técnica
- Setup de ambiente de desenvolvimento
- Implementação do backend básico
- Desenvolvimento da interface web

Documento gerado em: 30 de outubro de 2025
Versão: 2.0
Responsável: Arquitetura de Soluções